MODULE 24 · DOMAIN 4 - SECURITY AND COMPLIANCE

IAM Mechanics

Policy evaluation, roles and trust, permissions boundaries, condition keys and Access Analyzer.

DOMAIN 4 of 5 EXAM WEIGHT 16% MODULE 24 of 35

admin@sthithapragna.com

Every request asks one question: **is this principal allowed this action on this resource, in this context?** The evaluation order is the answer.

The policy types

TYPE	WHAT IT DOES
Identity-based	Attached to a user, group or role. Grants permissions to that identity
Resource-based	Attached to a resource, such as an S3 bucket or KMS key. Names a Principal
Permissions boundary	A ceiling on an identity. It grants nothing
Service control policy	An organisation-wide filter on accounts. It grants nothing
Resource control policy	An organisation-wide filter on resources. It grants nothing
Session policy	Passed when assuming a role. Further restricts that session

ON THE EXAM

Only the first two ever grant permission. The rest can only take it away, which is why adding a boundary never fixes an access denied.

How a request is evaluated

01

Explicit deny

Any explicit Deny anywhere ends it. Nothing overrides this.

›

02

Organization policies

SCPs and RCPs must allow, or the request stops here.

›

03

Resource-based policy

An Allow here can be sufficient on its own, within one account.

›

04

Identity-based policy

Must allow, unless a resource policy already granted it.

›

05

Boundary and session

A permissions boundary and any session policy must also allow.

ON THE EXAM

The default is deny. Something must actively allow, and nothing anywhere may explicitly deny. Both conditions, every time.

Explicit deny and implicit deny

- **An explicit Deny always wins.** It cannot be overridden by any Allow in any other policy, including an administrator policy.
- **An implicit deny** is simply the absence of an Allow. It is the default outcome and it is easy to fix.
- Diagnosing the difference matters: an implicit deny needs a grant added; an explicit deny needs something found and removed.
- Explicit denies with condition keys are how guardrails work. Deny unless MFA is present, deny unless encrypted in transit.

> ON THE EXAM
>
> "I gave them administrator access and it still fails" is an explicit deny somewhere. Look at SCPs, boundaries and resource policies, in that order.

Identity-based, resource-based, and the cross-account rule

- **Within one account**, a resource-based policy naming the principal directly can be **sufficient on its own**.
- **Across accounts, both sides must allow.** The resource policy in one account, and the identity policy in the other.
- This is the most common cause of a cross-account access denied: one side was configured and the other was forgotten.
- Not every service supports resource-based policies. S3, KMS, SQS, SNS, Lambda and Secrets Manager do; EC2 does not.

> ON THE EXAM
>
> Cross-account is an AND, not an OR. Granting generously on the bucket achieves nothing if the calling role has no matching permission.

Roles and trust policies

- A role has two policies: a **trust policy** saying **who may assume it**, and permissions policies saying **what it can do**.
- The trust policy **is a resource-based policy**. That is why it names a Principal while permissions policies do not.
- **AssumeRole** for AWS principals, **AssumeRoleWithWebIdentity** for OIDC, **AssumeRoleWithSAML** for federated workforce identities.
- Credentials are **temporary**. Default one hour, up to the role's maximum session duration, and up to 12 hours.

> ON THE EXAM
>
> "Cannot assume role" is the trust policy. "Assumed the role and then got AccessDenied" is the permissions policy. Two different files.

STS, sessions and role chaining

- **AssumeRole** returns temporary credentials. Default **1 hour**, up to the role's **MaxSessionDuration**, which can be **12 hours**.
- **Role chaining** - assuming a role from an already-assumed role session - is **capped at 1 hour**, whatever the role allows.
- **GetSessionToken** is for an IAM user needing MFA-authenticated credentials. **AssumeRole** is for everything else.
- **Session tags** passed at AssumeRole become **aws:PrincipalTag** values, which is what makes ABAC work with federation.

> ON THE EXAM
>
> "Our 12-hour session keeps expiring after an hour" is role chaining. The cap is not configurable and it is easy to hit accidentally.

The confused deputy, and how to prevent it

- A **confused deputy** is a trusted service or third party tricked into using its permissions on an attacker's behalf.
- For third parties, use an **ExternalId** in the trust policy. The third party must supply it, and only your value works.
- For AWS services, use **aws:SourceArn** and **aws:SourceAccount** conditions, so the service may only act for your resources.
- **aws:PrincipalOrgID** restricts a resource policy to principals inside your organisation, without listing account IDs.

> ON THE EXAM
>
> aws:PrincipalOrgID is the maintainable version of a long account list. New accounts are covered automatically; closed ones stop working.

Getting non-AWS identities in

MECHANISM	USE IT FOR
IAM Identity Center	Workforce SSO across many accounts. The default for people
SAML 2.0 federation	A corporate IdP federating directly into IAM roles
OIDC / web identity	CI/CD such as GitHub Actions, and mobile apps. No stored keys
IAM Roles Anywhere	Servers outside AWS, using X.509 certificates from your own PKI
Cognito identity pools	Application end users needing temporary AWS credentials

ON THE EXAM

All five end in a role and temporary credentials. None needs a long-lived access key, which is the point.

Service-linked roles

- A predefined role that a service creates in your account to act on your behalf. Its trust policy names that service and **cannot be edited**.
- **Service control policies do not affect service-linked roles.** That exception appears in exam questions.
- You usually cannot delete one until you have **removed the resources that depend on it**.
- Distinct from the **infrastructure roles** in module 21, which ECS and EKS create for you and which SCPs **do** govern.

> ON THE EXAM
>
> Two similar-sounding things behave oppositely: SCPs skip service-linked roles, and SCPs apply fully to infrastructure roles.

Users and access keys, where they survive

- **Long-lived access keys are the credential that leaks.** They appear in repositories, laptops and shared documents.
- A user may hold **two access keys at once**, which exists precisely to make rotation possible without an outage.
- Rotation: create the second key, update everything, **deactivate** the first, confirm nothing broke, then delete it.
- The **credential report** is the evidence: key age, last used, and whether MFA is enabled, for every user.

> ON THE EXAM
>
> Deactivate before delete. Deactivating is reversible in seconds; deleting is not, and something you forgot is always still using it.

Permissions boundary or service control policy?

PERMISSIONS BOUNDARY

A ceiling on one identity

- Attached to a specific user or role
- Effective permissions are the intersection with the identity policy
- Grants nothing by itself
- Lets you delegate role creation safely

SERVICE CONTROL POLICY

A ceiling on whole accounts

- Attached to an OU or account in Organizations
- Applies to every principal in those accounts
- Grants nothing by itself
- Does not apply to the management account

THE TIEBREAKER

Both are ceilings and neither grants. The boundary is per identity; the SCP is per account. The classic boundary use is safe delegation.

Permissions boundaries in practice

- **Effective permissions are the intersection** of the identity policy and the boundary. Both must allow.
- **A boundary never grants.** Attaching one to fix an access denied makes things worse, never better.
- The delegation pattern: allow a developer to create roles, but **only if they attach your boundary**, using an iam:PermissionsBoundary condition.
- That condition is what makes it safe. Without it, they simply create a role with no boundary and full permissions.

> ON THE EXAM
>
> The boundary alone is not the control. The condition requiring the boundary to be attached is the control.

Condition keys worth knowing

KEY	WHAT IT RESTRICTS
aws:PrincipalOrgID	To principals in your organisation, without listing accounts
aws:SourceArn / aws:SourceAccount	Which resource an AWS service may act on your behalf for
aws:MultiFactorAuthPresent	Requires MFA on the session
aws:SecureTransport	Requires TLS. Deny when false is the standard bucket guardrail
aws:PrincipalTag / aws:ResourceTag	Attribute-based access control, matching tags to tags
aws:RequestTag / aws:TagKeys	What tags may be applied at creation time
aws:SourceVpce / aws:SourceVpc	Access only through a named VPC endpoint or VPC

ON THE EXAM

aws:SourceIp does not behave as expected through a VPC endpoint, because the traffic never traverses the public internet. Use the VPC keys instead.

Attribute-based access control

- One policy compares a **tag on the principal** with a **tag on the resource**. Access is granted when they match.
- The policy never changes as teams and resources are added. Only the tags change.
- **Session tags** can be passed when assuming a role, so a federated user carries their department as a tag.
- It depends entirely on **tagging discipline**. Module 23's tag policies are what make ABAC trustworthy.

> ON THE EXAM
>
> Role-based access control needs a new policy per team. ABAC needs a tag. That is the whole argument, and it only works if tagging is enforced.

Two policy-writing traps

- **NotAction with Allow** grants everything except what you listed, including services that did not exist when you wrote it.
- **NotResource with Allow** is the same trap for resources. Both are usually a mistake outside a Deny statement.
- **Policy variables** such as ${aws:username} let one policy give each user access to their own prefix or their own resource.
- **Wildcards in Action** are worth checking: s3:Get* is broad, and s3:* is broader than most people intend.

> ON THE EXAM
>
> NotAction inside a Deny is a legitimate and useful pattern. NotAction inside an Allow is almost always wrong.

IAM Access Analyzer: three kinds of analyzer

- **External access**: which resources are shared outside your zone of trust. **No additional charge.**
- **Unused access**: roles, users, permissions and credentials with no activity in a window you choose. Charged per identity analysed.
- **Internal access**: who **inside** your organisation can reach selected business-critical resources. Newer, and charged.
- **These are separate analyzers.** Having one does not give you the others, and each must be created explicitly.

> ON THE EXAM
>
> External access analysis is free, so there is no reason not to have it. The other two cost money and answer different questions.

Access Analyzer beyond findings

- **Policy generation** builds a least-privilege policy from **CloudTrail history** of what an identity actually did.
- **Policy validation** checks a policy against grammar and best practice, with security, error, warning and suggestion findings.
- **Custom policy checks** answer specific questions in a pipeline: **does this change grant new access**, or **does it grant a forbidden action**?
- That makes it a **preventive** control in CI/CD, alongside CloudFormation Hooks from module 18.

> ON THE EXAM
>
> Custom policy checks in a pull request are the IAM equivalent of a change set: see what the policy would newly permit, before merging.

Auditing what already exists

- **Credential report**: an account-wide CSV of every user, their key ages, password age, and MFA status.
- **Last accessed data**: which services, and for some services which actions, an identity has actually used, and when.
- That is the evidence for removing permissions, and it works on users, roles, groups and policies.
- **Policy simulator** tests whether a specific principal would be allowed a specific action, without making the call.

> ON THE EXAM
>
> Last accessed data plus Access Analyzer's unused access findings are the two evidence sources for a least-privilege exercise.

The root user, centrally managed

- **Centralised root access management** removes root credentials from member accounts in an organisation entirely.
- Privileged actions are then performed with **sts assume-root**, producing a short-lived session scoped by a **task policy**.
- That eliminates hundreds of root passwords, MFA devices and recovery email addresses.
- **It does not apply to the management account**, whose root user must still be secured the traditional way.

> ON THE EXAM
>
> Only a specific short list of actions genuinely requires root, such as removing an S3 bucket policy that locked everyone out. Everything else has an IAM path.

The access denied ladder

- **1. Is there an explicit deny?** SCP, RCP, boundary, resource policy or session policy. Nothing overrides one.
- **2. Is this cross-account?** If so, check both sides. Both must allow independently.
- **3. Is a condition failing?** MFA, source VPC endpoint, tag mismatch, TLS. The error message often names the key.
- **4. Is it KMS?** Access to an encrypted resource needs permission on the key as well as the resource.

> ON THE EXAM
>
> The KMS rung catches people constantly. Reading an encrypted object needs s3:GetObject and kms:Decrypt, from both the identity and the key policy.

> A developer has the AdministratorAccess managed policy attached to their role. They attempt to launch an EC2 instance in eu-west-1 and receive an AccessDenied error. Launching in eu-west-2 succeeds.
>
> No permissions boundary is attached to the role.

What is the most likely cause?

A The AdministratorAccess policy does not cover EC2 in all Regions

B A service control policy explicitly denies actions outside approved Regions

C The role's identity policy needs an explicit Allow for eu-west-1

D The role's trust policy does not permit the developer

◆ PAUSE AND DECIDE

What is the most likely cause?

A **AdministratorAccess is Region-limited**

*It grants Action * on Resource *, with no Region condition. It is not scoped by Region at all.*

B **An SCP denying non-approved Regions**

An explicit deny overrides any Allow, including AdministratorAccess. Region restriction is the most common SCP, and it explains the asymmetry exactly.

C **Add an explicit Allow**

*Nothing overrides an explicit deny. Adding permissions to an identity that already has Action * changes nothing.*

D **Trust policy**

They assumed the role successfully and are receiving AccessDenied on an action, not on AssumeRole. Wrong policy entirely.

THE SIGNAL

Administrator access plus AccessDenied means an explicit deny, always. The Region asymmetry then names which guardrail it is.

> Account A's application role must read objects from a bucket in account B. The bucket policy in account B allows s3:GetObject for the role's ARN in account A. The role in account A has no S3 permissions in its identity policy.
>
> Requests fail with AccessDenied.

What must change?

A Nothing. A resource-based policy naming the principal is sufficient

B Add s3:GetObject on that bucket to the role's identity policy in account A

C Add the bucket owner's account to the role's trust policy

D Make the bucket public

◆ PAUSE AND DECIDE

What must change?

A Nothing needed

True within a single account. Across accounts both sides must allow independently, which is the rule being tested.

B Grant it in the identity policy too

Cross-account access is an AND. The bucket policy permits it and the calling identity must also be permitted to attempt it.

C Change the trust policy

Trust policies govern who may assume the role. Nobody is assuming anything here; the role is calling S3 directly.

D Make the bucket public

Grants the world access to solve a two-account problem. It also still would not help if the identity policy forbids S3.

THE SIGNAL

Option A is correct within one account and wrong across two. That single-account exception is exactly why this trips people up.

> A platform team must let application teams create IAM roles for their own workloads. Those roles must never be able to modify IAM, delete CloudTrail trails, or operate outside the approved Regions.
>
> The platform team does not want to review every role by hand.

What should the platform team implement?

A. A permissions boundary, plus an iam:PermissionsBoundary condition on the application teams' policies requiring it on any role they create

B. A permissions boundary attached to the application teams' own roles

C. An SCP denying iam:CreateRole in the application accounts

D. A Config rule detecting over-permissive roles after creation

◆ PAUSE AND DECIDE

What should the platform team implement?

A Boundary plus the required-boundary condition

The condition forces every role they create to carry the boundary, and the boundary caps what those roles can ever do. Delegation with a guarantee.

B Boundary on their own roles

Caps what the application teams themselves can do. It does nothing about the roles they create, which is the actual requirement.

C Deny CreateRole

Removes the delegation entirely. The requirement is that teams should be able to create roles, safely.

D Config rule after creation

Detective, so an over-permissive role exists until someone acts. The requirement is that it never can.

THE SIGNAL

The boundary is not the control. The condition requiring the boundary is. Option B has the right component and the wrong attachment point.

> A security team must produce evidence for auditors across 40 accounts: which IAM roles have not been used in the last 90 days, and which resources are shared with accounts outside the organisation.
>
> They want a single organisation-wide view and want to minimise cost where possible.

What should they configure?

A. An organisation-level unused access analyzer and an organisation-level external access analyzer

B. A single organisation-level analyzer, which covers both finding types

C. The IAM credential report in each account

D. AWS Config rules in each account

◆ PAUSE AND DECIDE

What should they configure?

A Two analyzers, both organisation-level

The finding types come from separate analyzers. External access is free; unused access is charged per identity, so this is also the cheapest correct answer.

B One analyzer for both

The plausible assumption, and wrong. Creating an external access analyzer does not produce unused access findings; each type needs its own.

C Credential reports

Covers users and their credentials, not roles, and it is per account rather than organisation-wide.

D Config rules

Evaluates resource configuration. It does not compute unused access or analyse resource policies for external sharing.

THE SIGNAL

Access Analyzer is three separate analyzers sharing a name. Assuming one covers everything is the trap, and the pricing difference reinforces the split.

> An application role can list objects in an encrypted S3 bucket but receives AccessDenied when downloading them. The role's identity policy allows s3:GetObject and s3:ListBucket on the bucket. The bucket uses SSE-KMS with a customer managed key.
>
> There are no service control policies in the account.

What is missing?

A kms:Decrypt permission, in both the role's identity policy and the KMS key policy

B s3:GetObjectVersion on the bucket

C A bucket policy allowing the role

D Block Public Access must be disabled on the bucket

◆ PAUSE AND DECIDE

What is missing?

A kms:Decrypt, on both sides

Listing needs no decryption; downloading does. KMS keys have a key policy, so the identity and the key must both permit it.

B s3:GetObjectVersion

Needed only when requesting a specific version. Ordinary downloads use s3:GetObject, which is already granted.

C A bucket policy

Same-account access works from the identity policy alone. A bucket policy is not required here.

D Disable Block Public Access

Governs public access. The role is an authenticated principal in the same account, so it is irrelevant.

THE SIGNAL

List works and get fails is the encryption signature. Listing reads metadata; retrieving content requires decrypting it.

> A GitHub Actions pipeline deploys to AWS using an IAM user's long-lived access keys stored as repository secrets. A security review requires that no long-lived AWS credentials exist outside AWS.
>
> The pipeline must continue deploying with no manual credential handling.

What should the team implement?

A. An IAM OIDC identity provider for GitHub, with a role the workflow assumes via web identity federation

B. Rotate the access keys automatically every 24 hours with a Lambda function

C. Store the access keys in Secrets Manager and have the pipeline fetch them

D. IAM Roles Anywhere with certificates issued to the GitHub runners

◆ PAUSE AND DECIDE

What should the team implement?

A OIDC federation with GitHub

GitHub presents a signed token, AWS exchanges it for temporary credentials. No AWS credential exists outside AWS at any point.

B Rotate keys every 24 hours

A long-lived key that changes daily is still a long-lived key, and a leaked one is valid until the next rotation.

C Keys in Secrets Manager

The pipeline needs credentials to read Secrets Manager, so you still store an AWS credential in GitHub. It moves the problem.

D IAM Roles Anywhere

Built for servers outside AWS using your own PKI. GitHub-hosted runners already provide an OIDC token, so certificates are unnecessary machinery.

THE SIGNAL

Option C is the circular one worth spotting: fetching a secret requires a credential, so the credential you were trying to remove is still there.

The five things to carry forward

- **Only identity and resource policies grant.** Boundaries, SCPs, RCPs and session policies only restrict.
- **Administrator access plus AccessDenied means an explicit deny.** Find it and remove it; granting more will not help.
- **Cross-account is an AND.** Within one account, a resource policy naming the principal can be enough on its own.
- **The boundary is not the control; the condition requiring it is.**
- **List works, get fails, on an encrypted bucket** means kms:Decrypt, from the identity and the key policy.

> ON THE EXAM
>
> Next: multi-account - Organizations, SCPs, IAM Identity Center and delegated administration.